Virtual Private Network (VPN) Lab

October 27, 2026

1. What this lab is about

A VPN (Virtual Private Network) is a private tunnel through an unsafe network such as the Internet. Anything that goes into the tunnel is hidden from outsiders, and only the right people are let in.

In this lab, I built a small VPN called miniVPN on virtual machines. The final version does 4 things:

1. It hides the traffic (encryption),
2. It proves the server is the real one (certificate check),
3. It checks the user's password,
4. It lets more than one user connect at the same time.

The machines used

Machine	Role	IP address(es)	Network
Host U	VPN client (the user)	10.0.2.3	NAT Network
Gateway	VPN server	10.0.2.4 and 192.168.60.1	NAT Network + Internal Network
Host V	Computer inside the private network	192.168.60.101	Internal Network
Host-U2	Second VPN client (Task 6 only)	10.0.2.5	NAT Network

Inside the tunnel, the server uses the address 192.168.53.1, and the clients get 192.168.53.5 and 192.168.53.6. The server name used for the certificate is vpnlabbinhr.com (which contains my last name, as the lab requires).

2. Task 1: VM setup

Goal: build the network.

Host V sits in a private network that is only connected to the Gateway. This means Host U should not be able to reach Host V directly. The VPN is what will let it in.

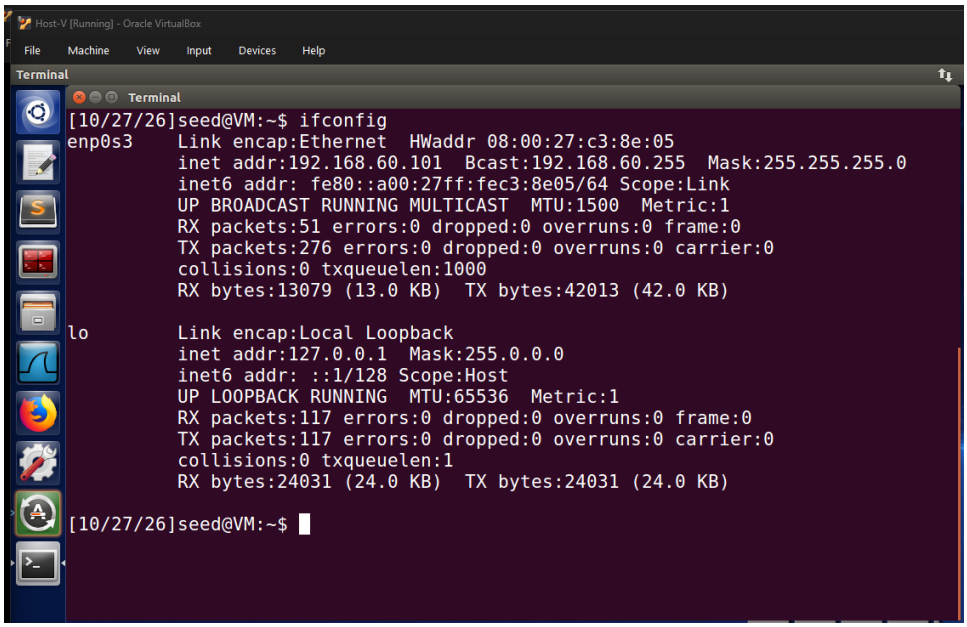


Figure 1: Host V has the private address 192.168.60.101.

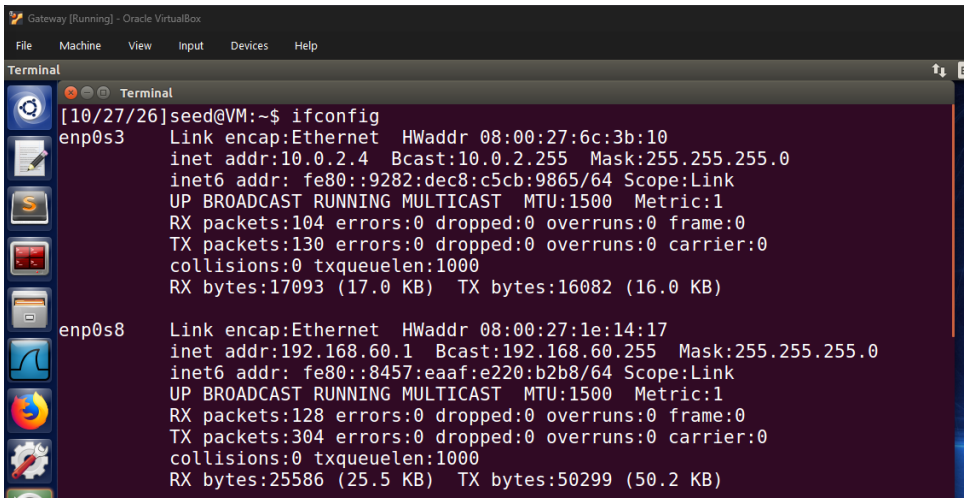


Figure 2: The Gateway has two network cards: 10.0.2.4 (towards Host U) and 192.168.60.1 (towards Host V).

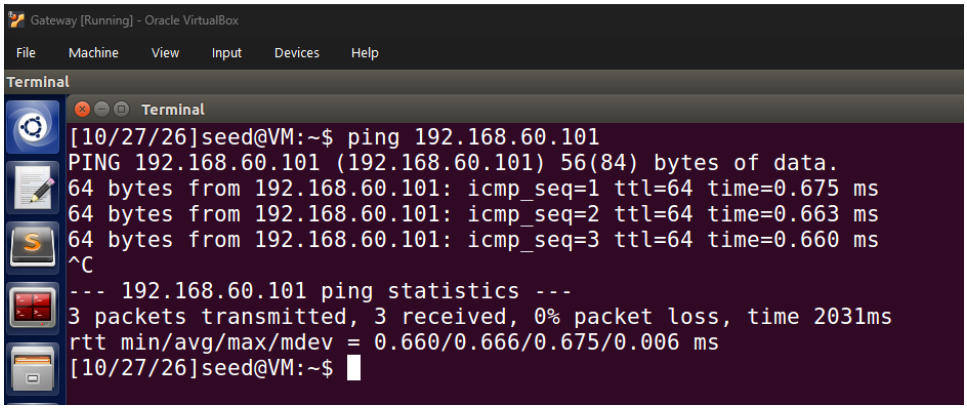


Figure 3: The Gateway can ping Host V. The private link works.

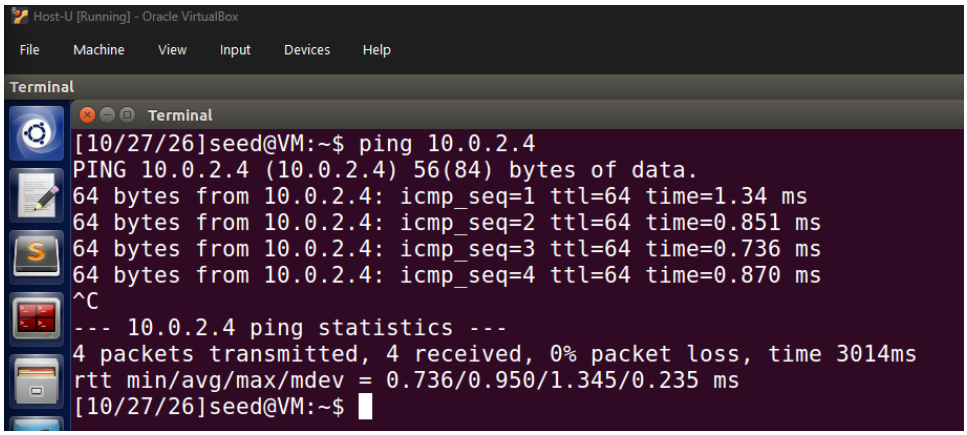


Figure 4: Host U can ping the Gateway at 10.0.2.4. The outside link works.

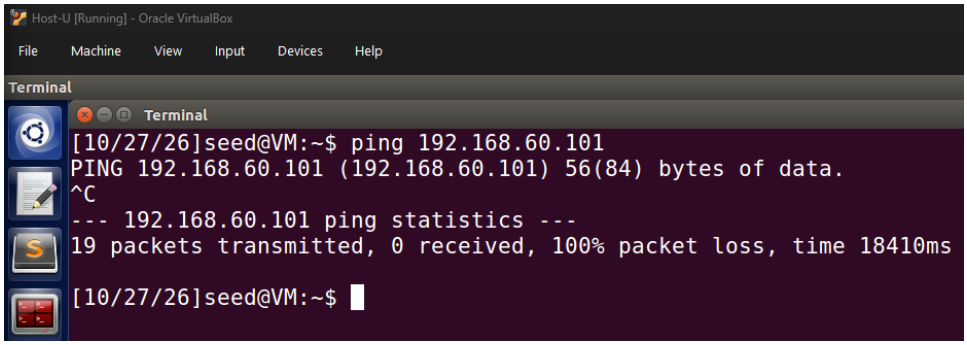


Figure 5: Host U cannot reach Host V (100% packet loss). This is the expected result: without the VPN, the private network is closed to Host U.

3. Task 2: Creating a VPN tunnel (no encryption yet)

A tunnel is made with a "TUN" device. This is a virtual network card called tun0. When the computer sends a packet to tun0, the VPN program picks it up, puts it inside a UDP packet, and sends it to the other end. The other end takes the packet out and hands it to its own tun0. This 1st version has no encryption.

Steps 1 and 2: Start the server and the client

On the Gateway, I started the server program, gave tun0 the address 192.168.53.1, and turned on IP forwarding. This lets the Gateway pass packets between the tunnel and the private network.

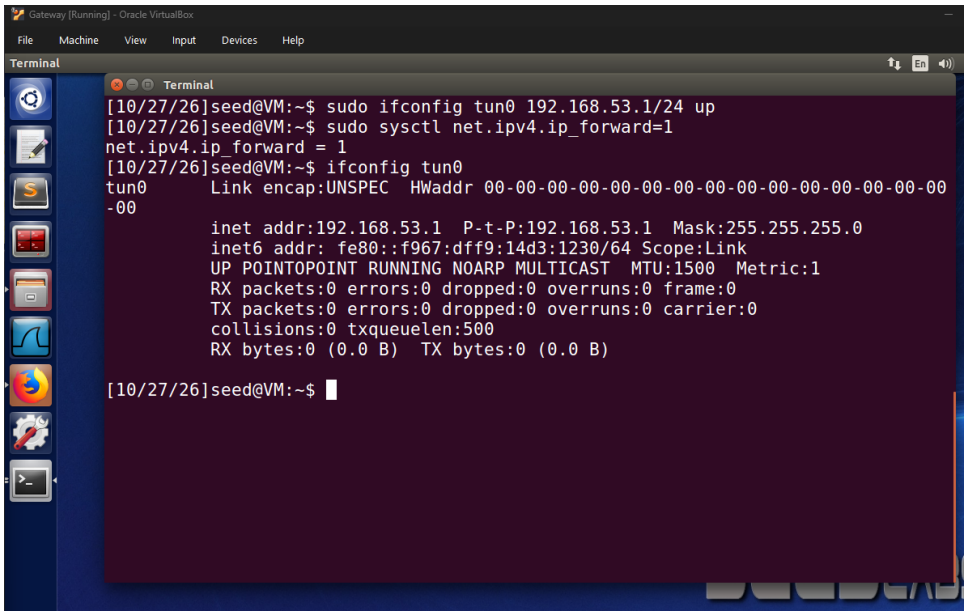


Figure 6: Gateway: tun0 has 192.168.53.1, and IP forwarding is on (value 1).

On Host U, I started the client program and gave tun0 the address 192.168.53.5. I also added a route that sends everything for the private network (192.168.60.0/24) into the tunnel.

A problem I fixed: the sample client had the server address fixed inside the code as 127.0.0.1, which is the computer itself, so nothing reached the Gateway. I changed it to the Gateway's address, 10.0.2.4.

Step 3: Routing on the client and the Gateway

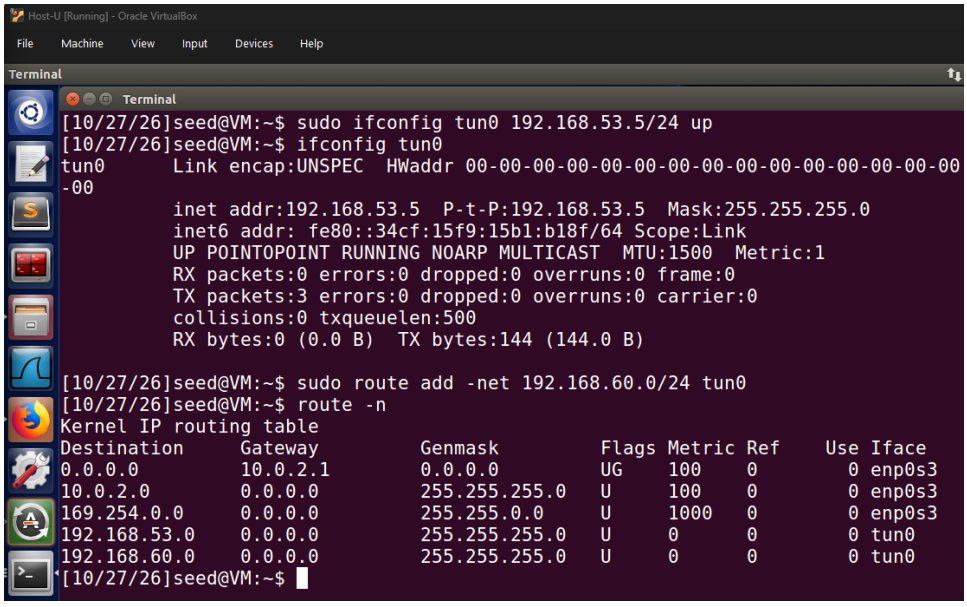


Figure 7: Host U: tun0 is 192.168.53.5, and both 192.168.53.0 and 192.168.60.0 go through tun0.

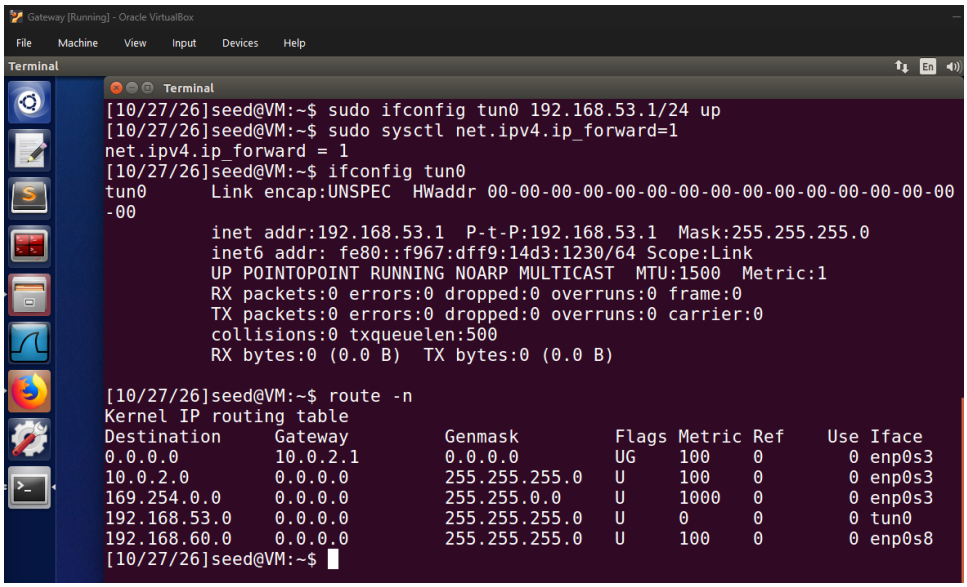


Figure 8: Gateway: 192.168.53.0 goes through tun0 (the tunnel) and 192.168.60.0 goes through enp0s8 (the private network).

Step 4: Routing on Host V

When Host U pings Host V through the tunnel, the packet arrives at Host V with the source address 192.168.53.5. Host V must send its reply back through the Gateway, so I added a route for 192.168.53.0/24 with the Gateway 192.168.60.1 as the next stop.

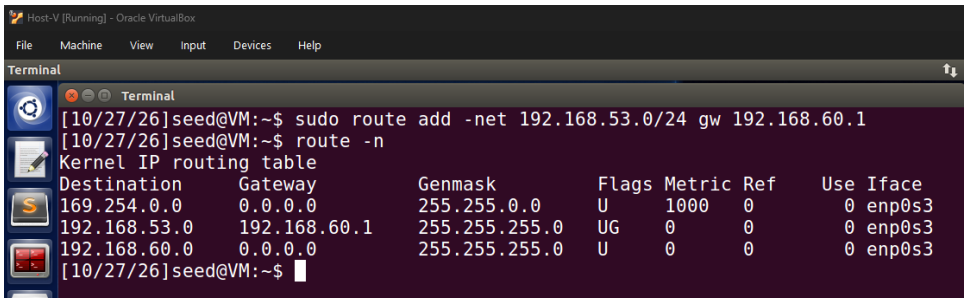


Figure 9: Host V: replies for 192.168.53.0/24 are sent to the Gateway (192.168.60.1).

Step 5: Testing the tunnel

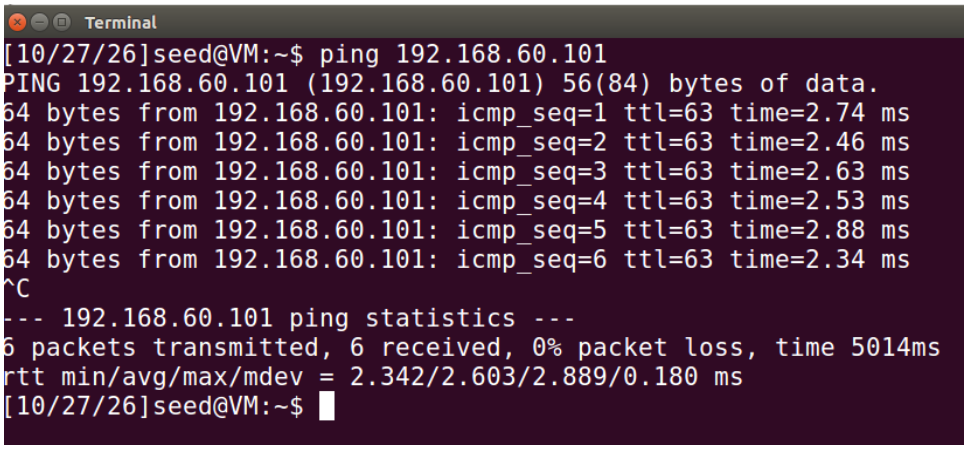


Figure 10: Host U can now ping Host V. Compare with Figure 5, where it failed before the VPN.

Which packets are tunnel traffic? On the real network card (enp0s3) the only packets are UDP packets between 10.0.2.3 and 10.0.2.4 on port 55555. These are the tunnel. Inside each of them is the original ping.

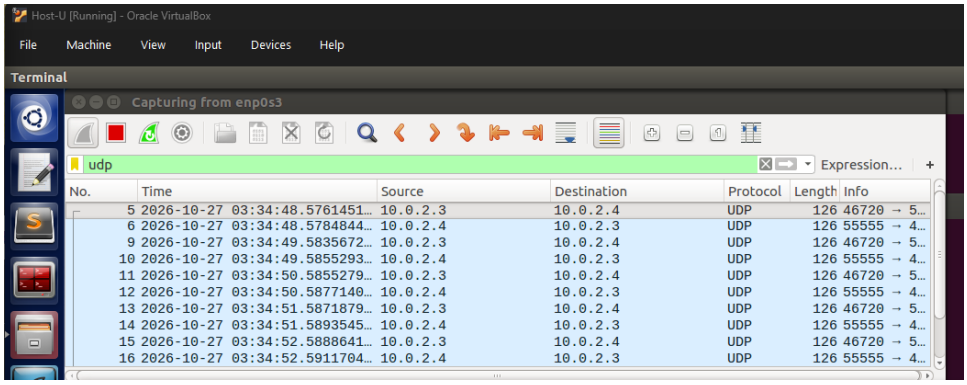


Figure 11: Tunnel traffic on Host U's network card enp0s3: UDP packets between 10.0.2.3 and 10.0.2.4 (port 55555). Each is 126 bytes.

The original ping packets (ICMP) are visible only on tun0. They are not tunnel traffic. They are the data that goes into the tunnel. The ping is 84 bytes. After it is wrapped in UDP, an IP header and an Ethernet header it becomes 126 bytes, which explains the difference.

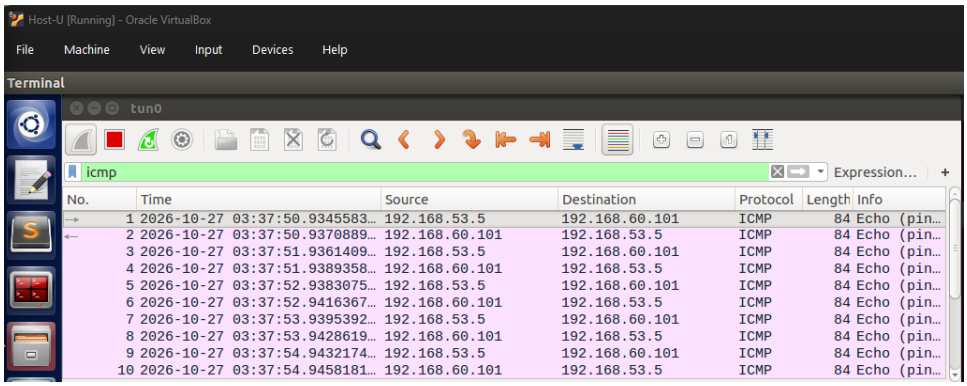


Figure 12: Not tunnel traffic: the original ping packets on tun0, between 192.168.53.5 and 192.168.60.101 (84 bytes each).

Telnet gives the same result.

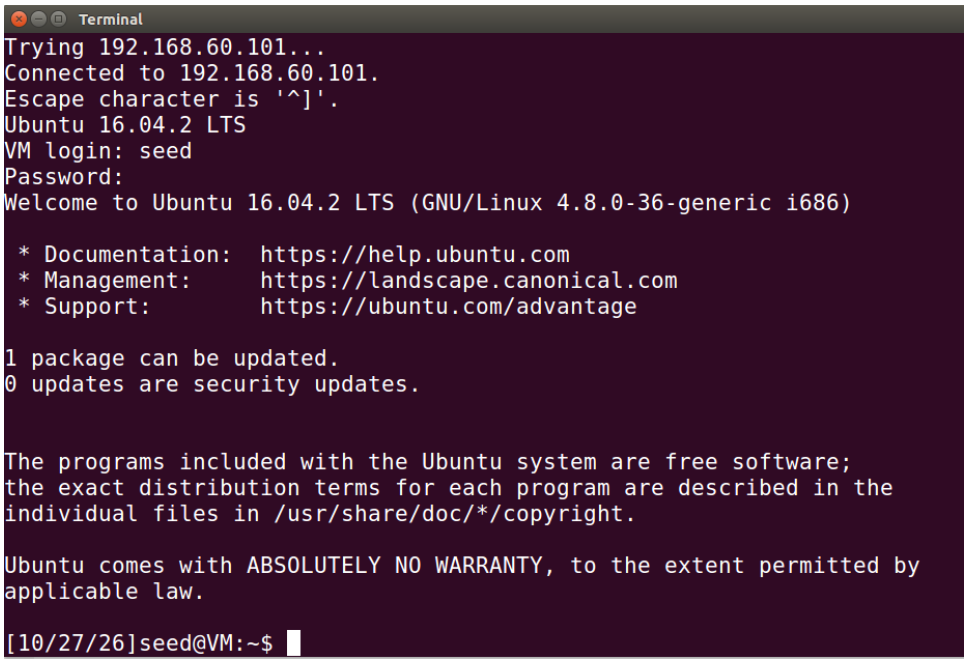


Figure 13: Telnet from Host U to Host V works through the tunnel.

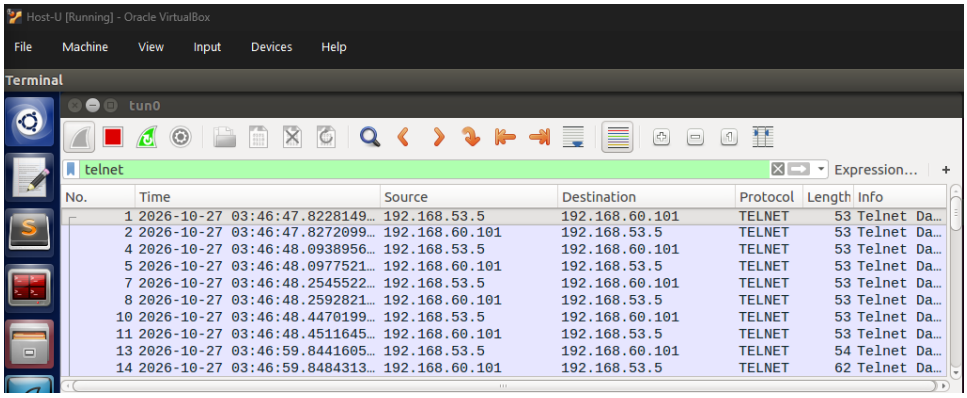


Figure 14: The original telnet packets on tun0. They are unprotected here, which is why encryption is added in Task 3.

Step 6: Tunnel-breaking test

While a telnet session was open, I stopped the VPN client to break the tunnel and typed some text.

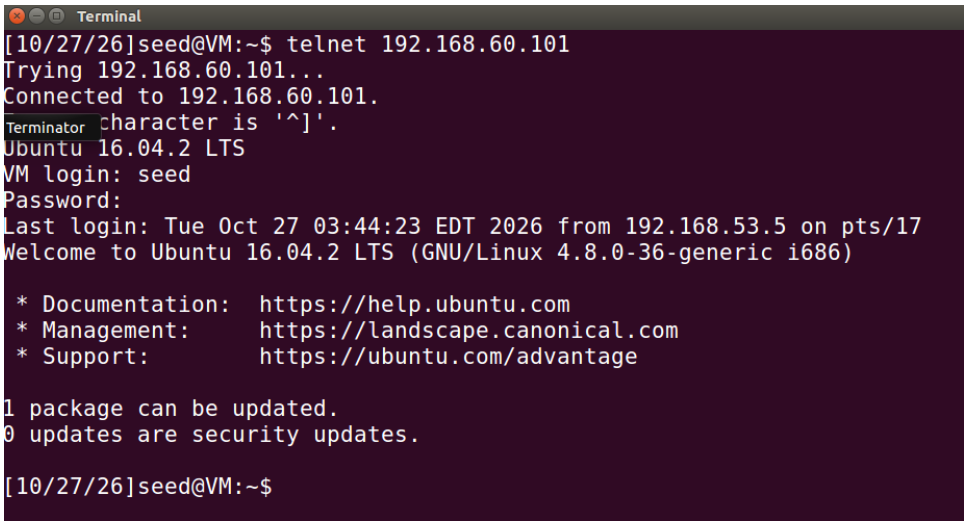


Figure 15: Tunnel stopped: the text typed in the telnet window did not appear, but the session stayed open.

Then I started the tunnel again. The text I typed earlier appeared and Host V answered it. The telnet session carried on as if nothing happened.

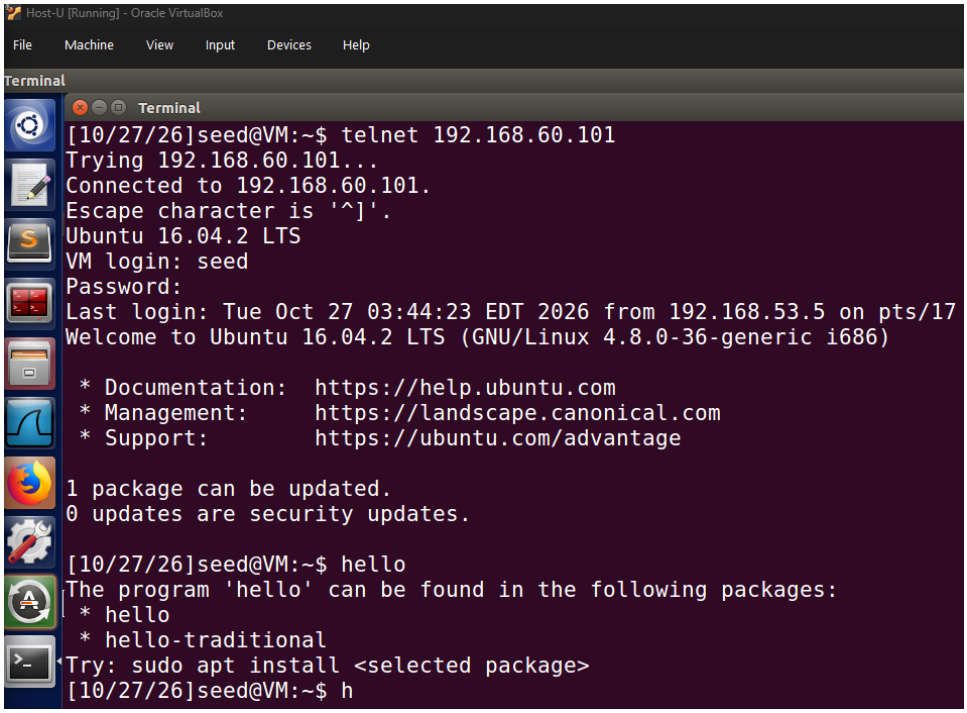


Figure 16: Tunnel restarted: the earlier "hello" went through and the session continued.

Why: the telnet connection exists only inside Host U and Host V, not inside the tunnel. The tunnel is just the road between them. While the road was closed, Host U kept re-sending the typed letters. When the road opened again, the letters got through and the connection continued. If the tunnel stays down too long, the computers give up and the connection breaks for good.

Extra test. Large packets

I pinged Host V with a 3500-byte packet (ping -s 3500). It is too big to fit in one packet (the limit is about 1500 bytes), so it gets segmented into pieces. This happens twice. First, the computer segments it into 3

pieces for tun0. Then each piece is wrapped in a new UDP packet, and those packets are still too big, so they are segmented again. That is why Wireshark shows packets of 1514 bytes and small ones of 62 bytes. The gateway reassembles the pieces, so all 3 pings succeed with no loss.

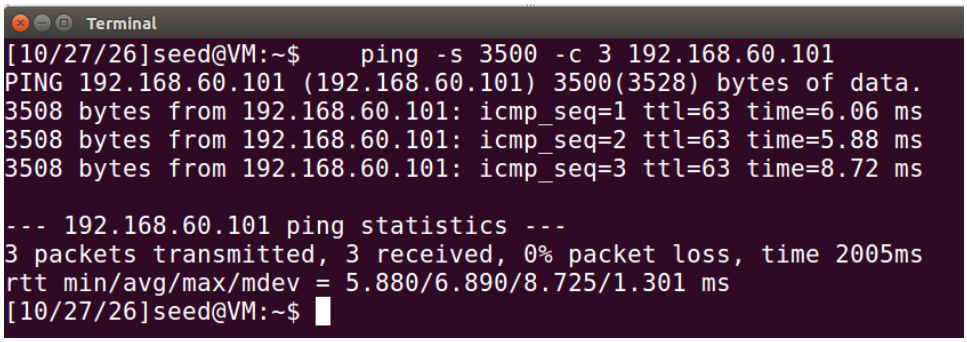


Figure 17: 3500-byte pings: 3 sent, 3 received, 0% loss.

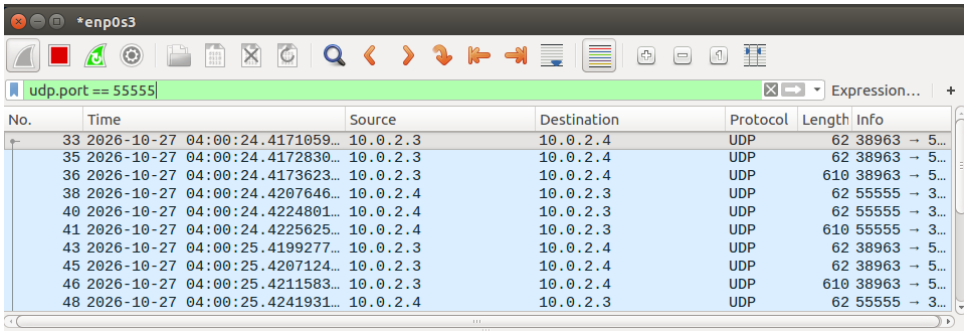


Figure 18: enp0s3: the tunnel packets (UDP port 55555). Lengths of 62 and 610 bytes are the segmented pieces.

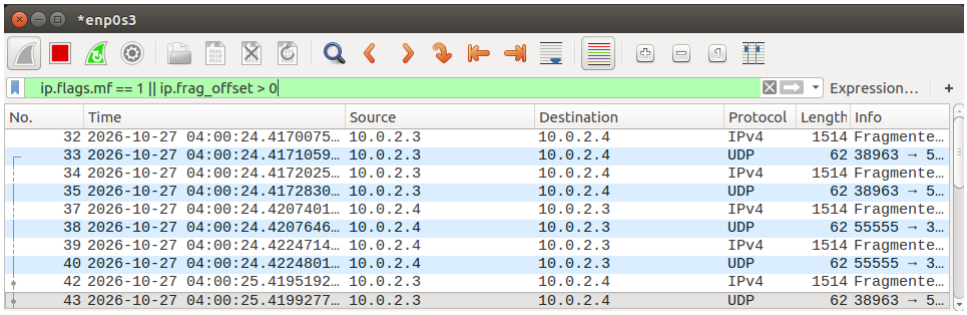


Figure 19: enp0s3: the pieces (fragments), including the full-size 1514-byte ones.

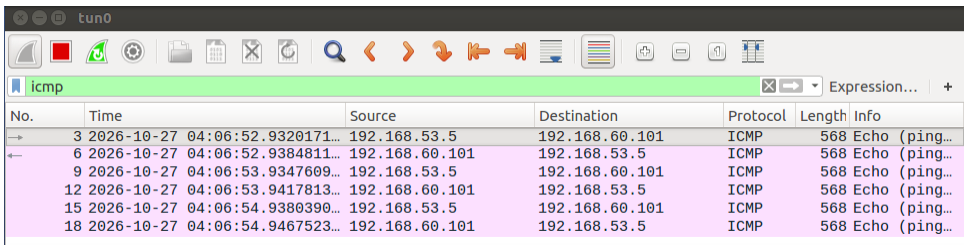


Figure 20: tun0: each ping appears once, as one 568-byte packet (Wireshark shows the put-together result).

4. Task 3: Encrypting the tunnel

The tunnel from Task 2 can be read by anyone who captures it. To fix this, I replaced UDP with TCP and wrapped the connection in TLS, which hides the content and detects tampering. The server listens on port 4433. The certificate used for this is created in Task 4 below.

```
[10/27/26]seed@VM:~/miniVPN$ cd ~/miniVPN
[10/27/26]seed@VM:~/miniVPN$ sudo ./vpnserver
TLS connection established with 10.0.2.3
```

Figure 21: Gateway: the server accepted a TLS connection from 10.0.2.3 (Host U).

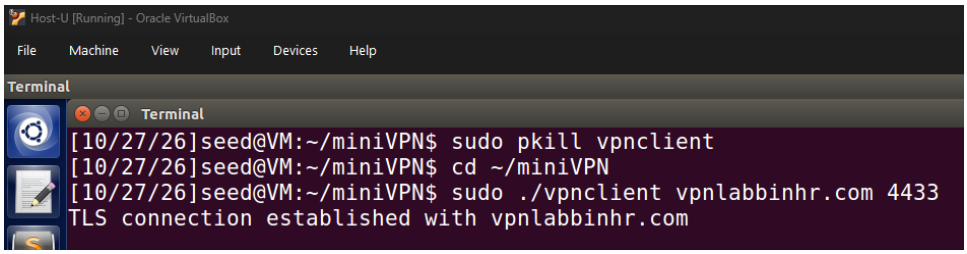


Figure 22: Host U: the client connected to vpnlabbinhr.com over TLS.

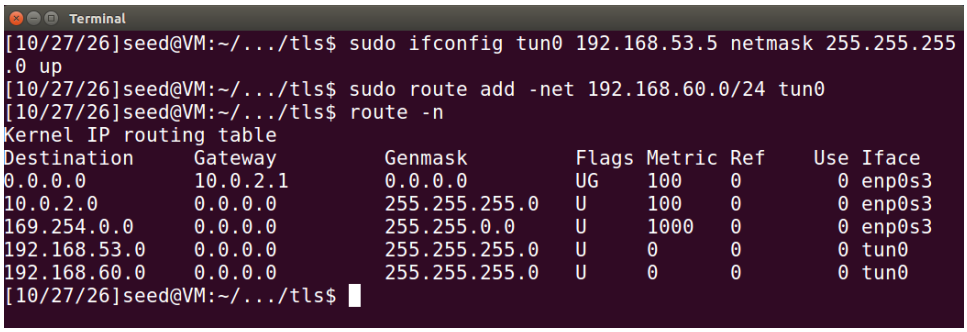


Figure 23: Host U: tun0 and the route to the private network are set up.

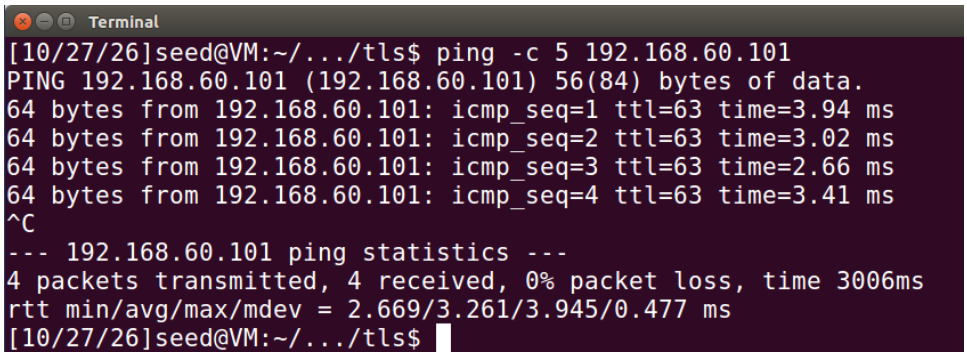


Figure 24: A ping from Host U to Host V works through the encrypted tunnel.

To show the traffic is encrypted, I configure Wireshark to identify port 4433 as TLS. It now labels the packets "TLSv1.2 Application Data". The content shows only unreadable characters.

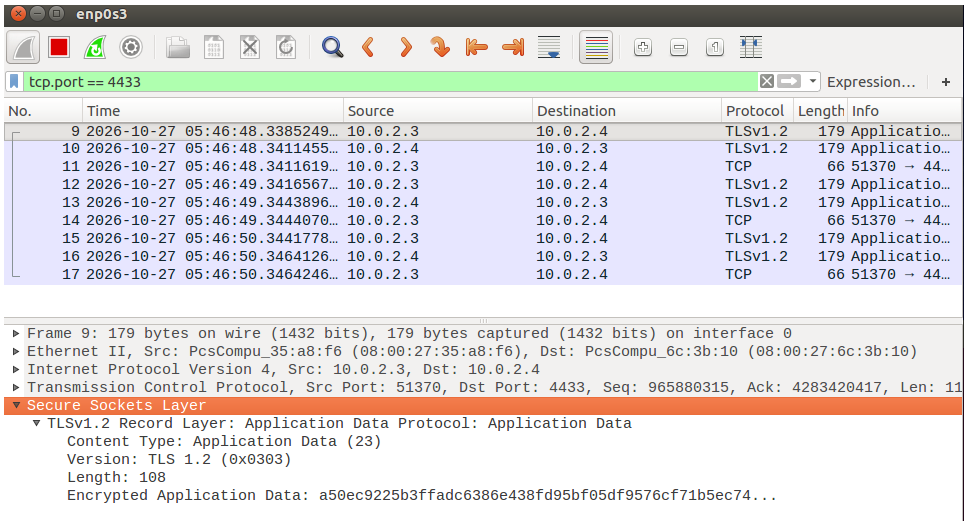


Figure 25: enp0s3: the tunnel packets are TLSv1.2. The "Encrypted Application Data" is unreadable, so the ping inside cannot be seen.

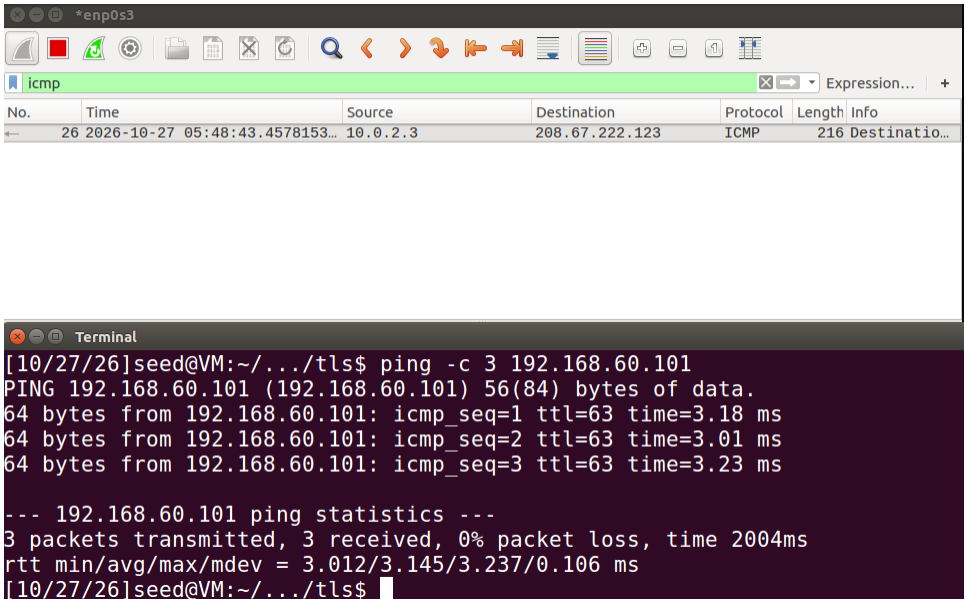


Figure 26: Filter "icmp" on enp0s3: none of the pings to 192.168.60.101 can be seen. The one line shown is an unrelated message to a different address (208.67.222.123).

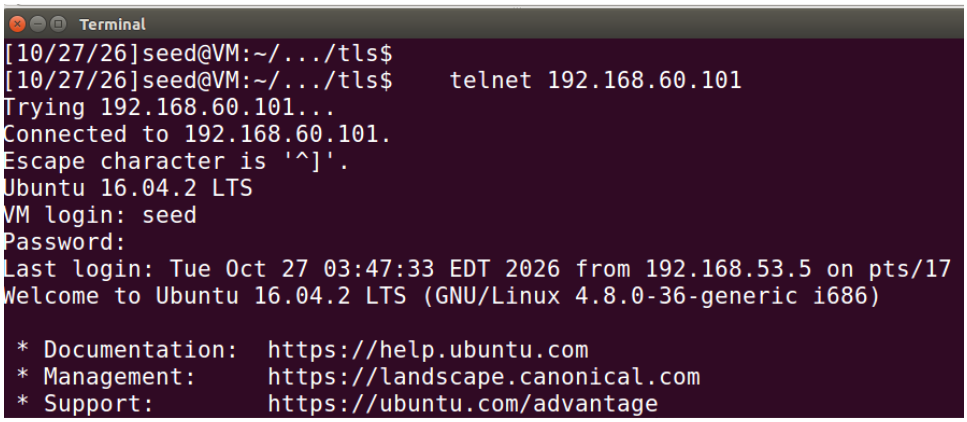


Figure 27: Telnet from Host U to Host V works through the encrypted VPN.

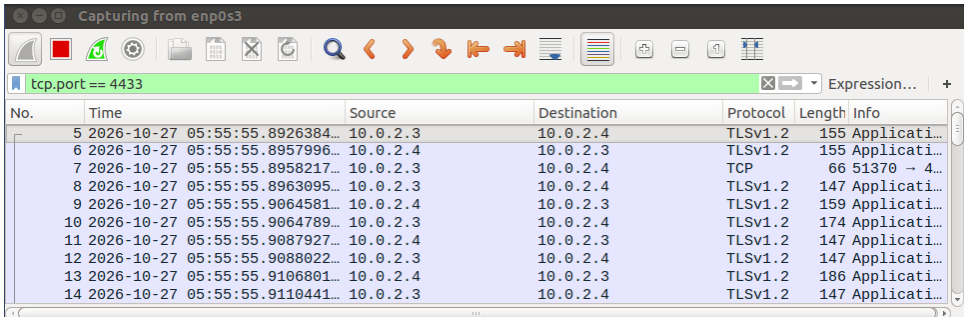


Figure 28: enp0s3 during telnet: only TLS packets, nothing readable.

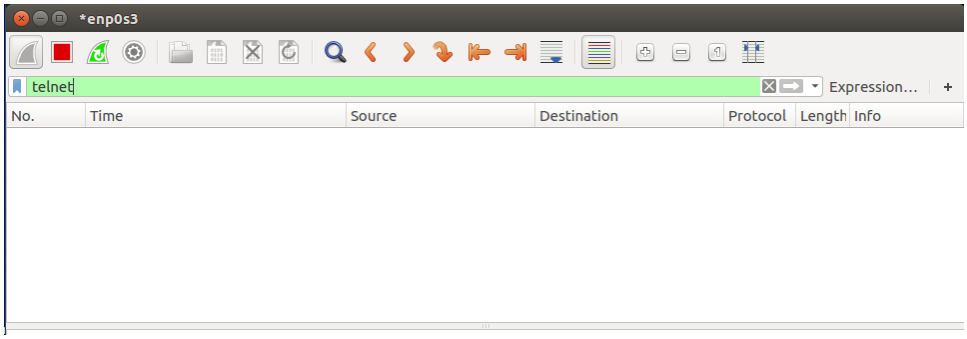


Figure 29: Filter "telnet" on enp0s3: nothing appears, because telnet is hidden inside TLS.

5. Task 4: Authenticating the VPN server

Before a user sends anything, the client must be sure it's talking to the real server and not an impostor. This works with certificates. A trusted authority (a CA) signs the server's certificate, and the client checks that signature.

The certificate that came with the lab sample expired in 2019, and the lab asks us to make our own CA anyway. So I made my own CA ("Binhr CA") and a certificate for the server name vpnlabbinhr.com.

Step 1: Create my own CA

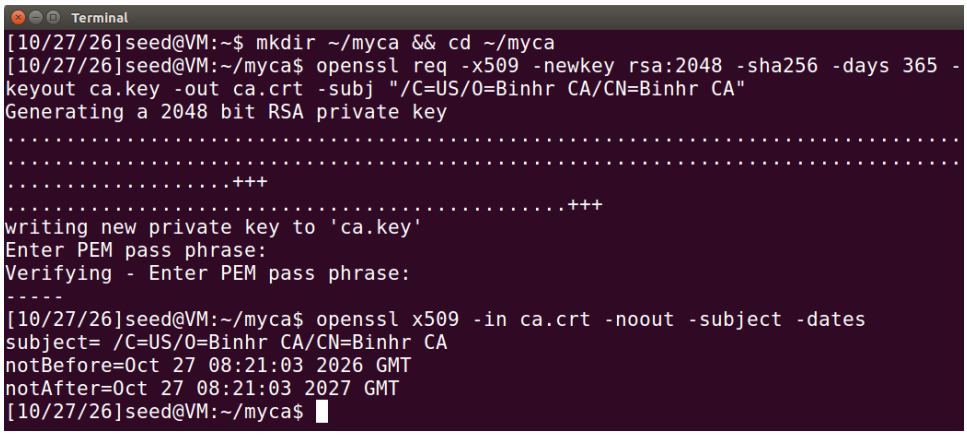


Figure 30: The CA certificate was created. It is valid for one year.

Step 2: Create and sign the server certificate



Figure 31: The server certificate for vpnlabbinhr.com was signed by my CA. "server.crt: OK" means the signature is correct.

Step 3: Install the certificate on the server

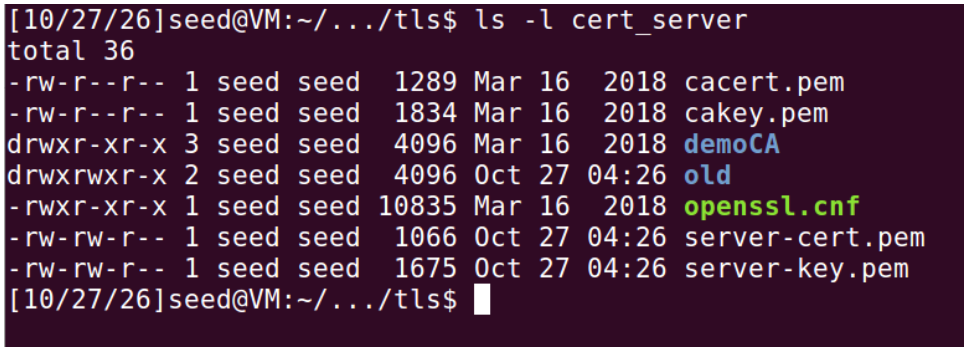


Figure 32: Gateway: server-cert.pem and server-key.pem are in the cert_server folder. The server loads them from here.

Step 4: Let the client trust my CA

I copied my CA certificate to the client's ca_client folder and made a link named by its hash (3c0f72a8.0), which is how the TLS library finds it. I also made vpnlabbinhr.com point to the Gateway (10.0.2.4) in the hosts file.

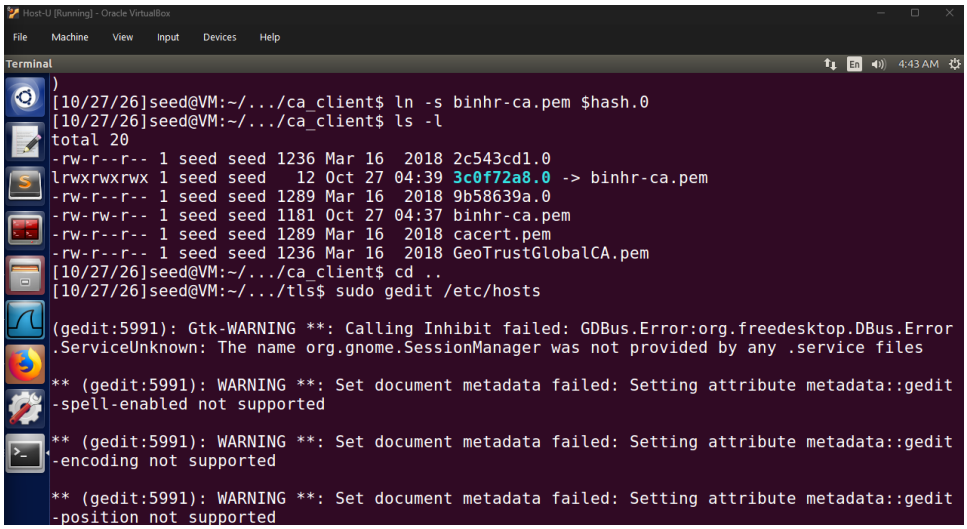


Figure 33: Host U: the CA file binhr-ca.pem and its hash link 3c0f72a8.0 in the ca_client folder.

Step 5: Right name versus wrong name

The client checks that the name in the certificate matches the name the user typed. Using the wrong name for the same server must fail. The first command below used the wrong name (vpnlabwrong.com) and was refused. The second used the correct name and worked (cipher AES256-GCM-SHA384).

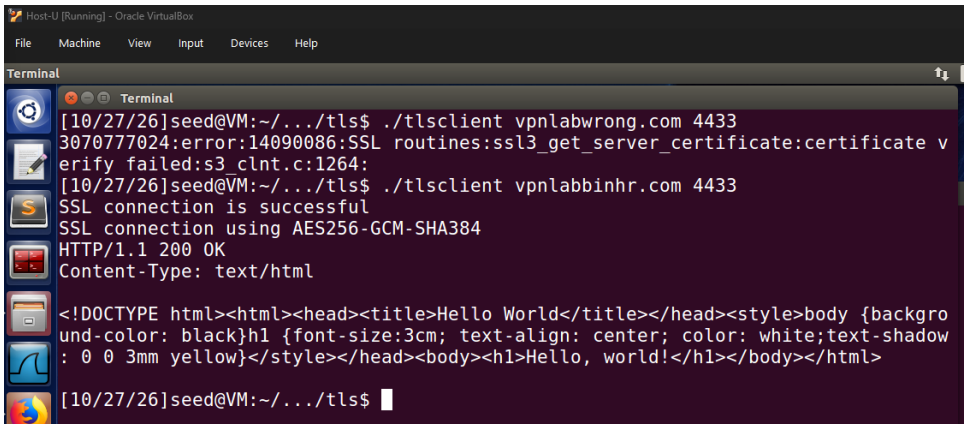


Figure 34: Wrong name: "certificate verify failed". Correct name: connection successful.

Step 6: Fake server (man-in-the-middle) test

To simulate an attacker, Host-U2 ran a fake server with a certificate that my CA never signed. On Host U, I made vpnlabbinhr.com point to the attacker (10.0.2.5). The client rejected the connection and never asked for the username or password, so nothing secret was sent to the attacker.

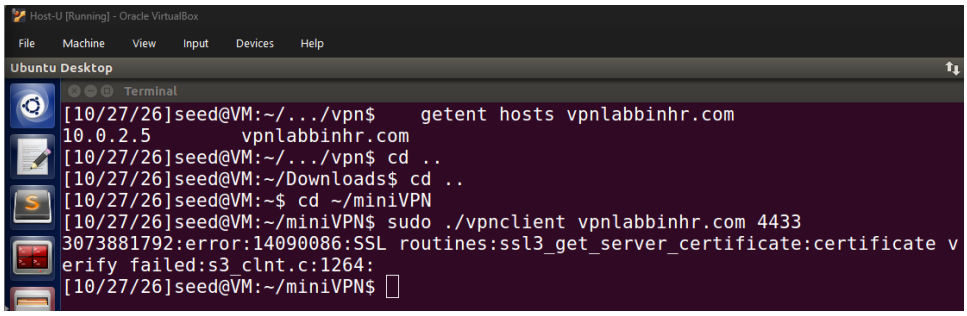


Figure 35: Host U: vpnlabbinhr.com now points to 10.0.2.5 (the attacker). The VPN client stops with "certificate verify failed".

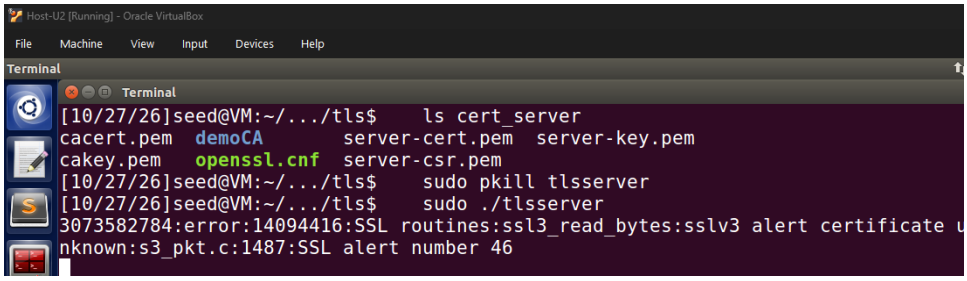


Figure 36: The attacker's side: the fake server got an "unknown certificate" alert from the client.

6. Task 5: Authenticating the VPN client

Only people with an account on the server may use the VPN. After the TLS connection starts, the client asks for a username and password and sends them through the encrypted connection. The server checks them against its account list (the shadow file). The password is typed when the program runs which is hidden on screen (getpass), and its not written in the code.

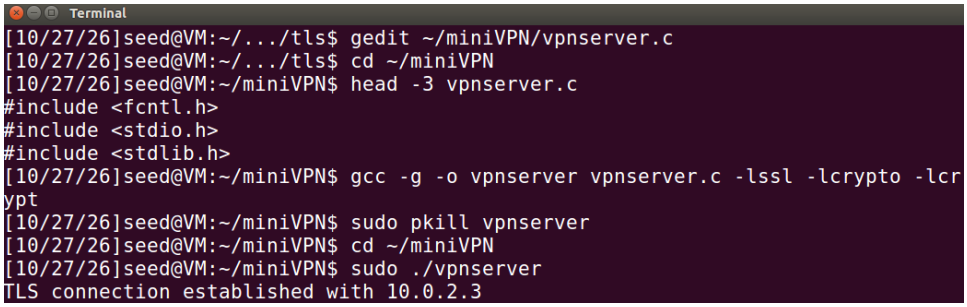


Figure 37: Gateway: the server was built with the login check and is running.

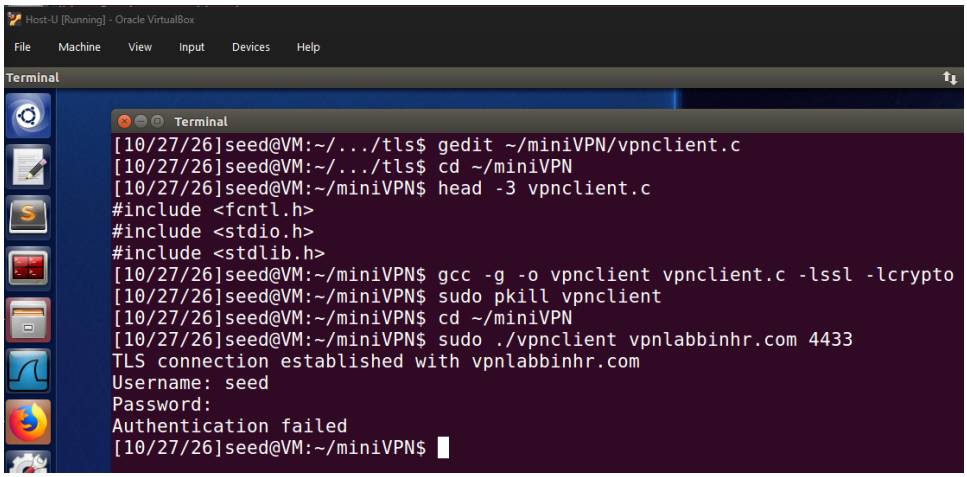


Figure 38: Host U: a wrong password is refused ("Authentication failed") and no tunnel is created.

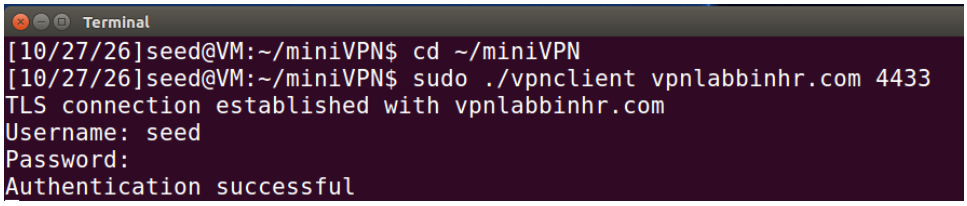


Figure 39: Host U: the correct password gives "Authentication successful".

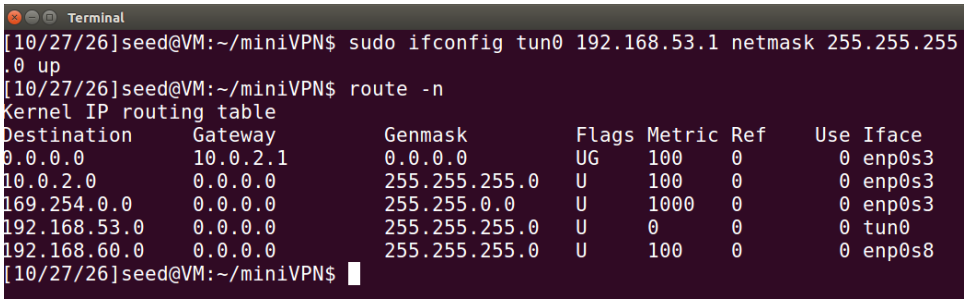


Figure 40: Gateway after the login: tun0 has the tunnel network and the private network is reachable.

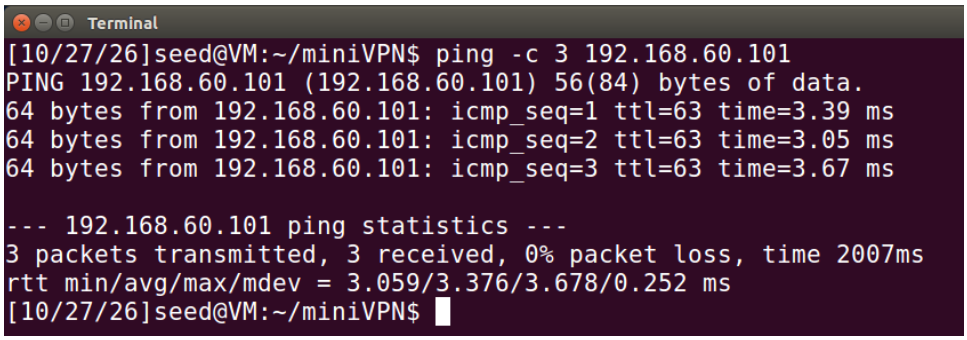


Figure 41: After logging in, the ping to Host V works.

7. Task 6: Supporting multiple clients

The server now handles several users at once. The main (parent) process waits for new users. For each new user, it starts a child process that handles that user's TLS connection and login, and it gives the user their own tunnel address (192.168.53.5 for the first, 192.168.53.6 for the second).

Data going into the tunnel is simple: each child writes the packets from its user to tun0. Data coming out is harder. When a packet arrives at tun0, the parent reads its destination address (for example, 192.168.53.6), decides which user it belongs to, and passes it to that user's child process, which sends it through that user's tunnel. The parent and children pass the packets to each other through an inter-process channel (a socket pair, which keeps each packet separate).

For the test, I made a second client machine, Host-U2.

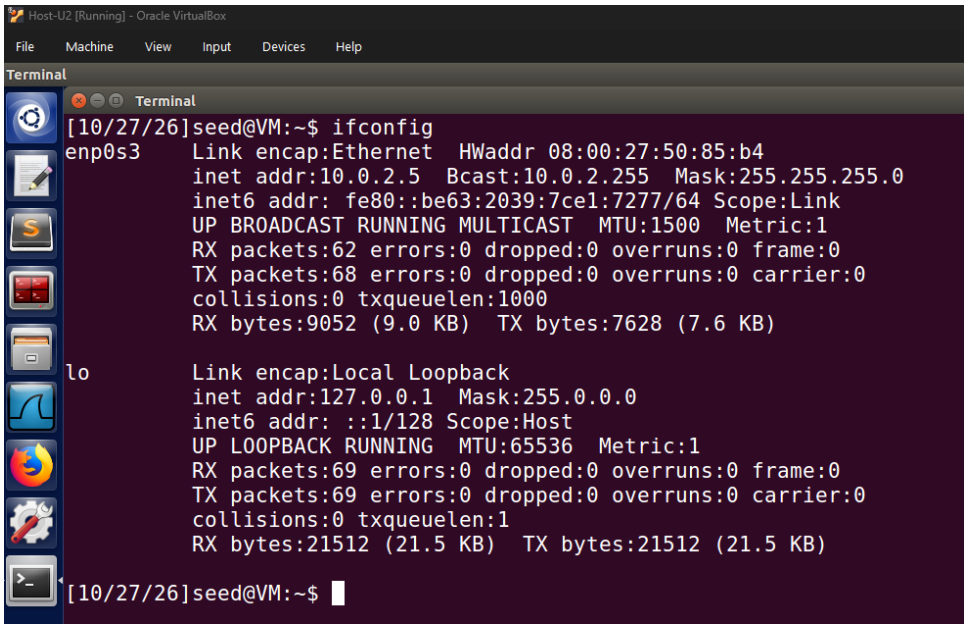


Figure 42: Host-U2, the second client, has its own address: 10.0.2.5.

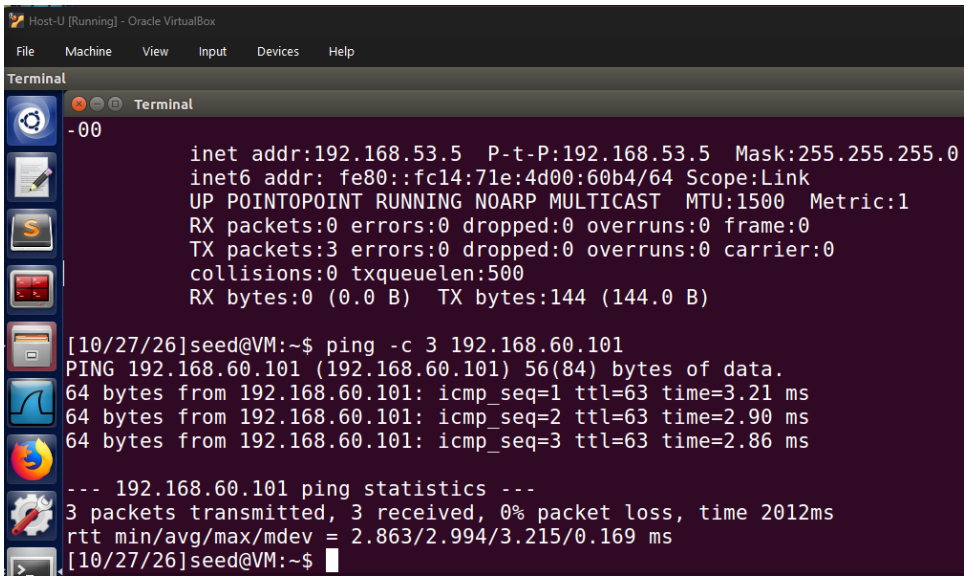


Figure 43: Client 1 (Host U): tun0 was set up automatically with 192.168.53.5, and the ping works.

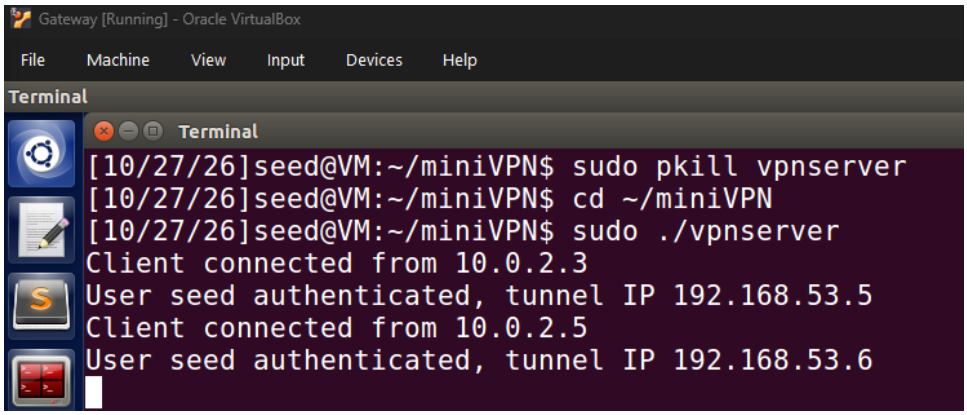


Figure 44: Gateway: both clients are connected, with tunnel addresses 192.168.53.5 and 192.168.53.6.

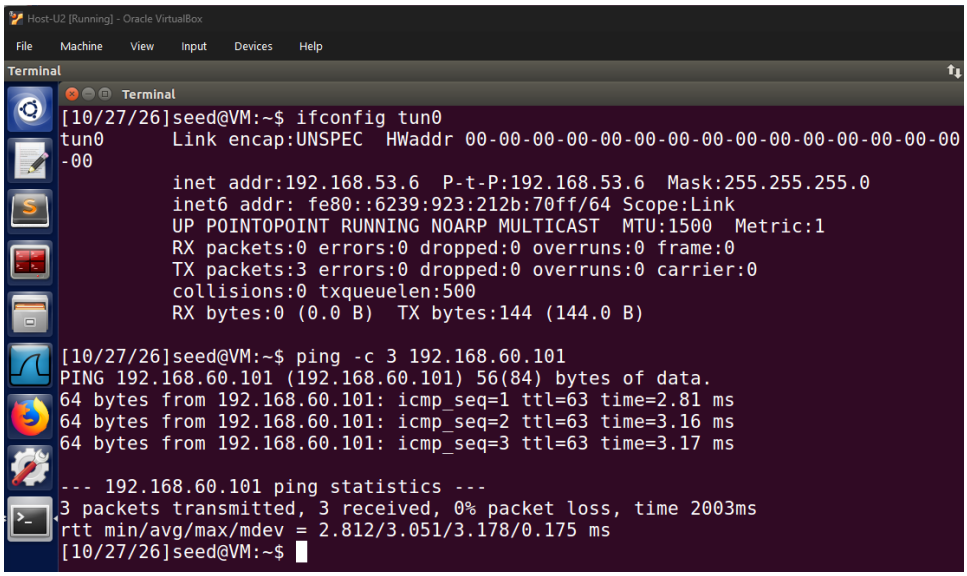


Figure 45: Client 2 (Host-U2): tun0 is 192.168.53.6 and the ping works.

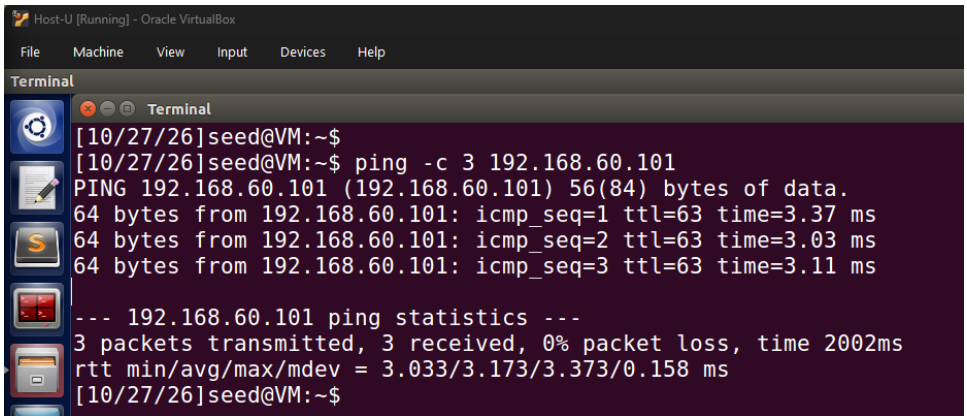


Figure 46: Host U pings again while both clients are connected. The first tunnel still works.

8. Checklist: state after rebooting

The demo checklist asks to reboot the machines and show the reboot time next to the current time. Host U and the Gateway are within 5 minutes. Host V had to be rebooted a second time to make its private address survive a restart, so its gap is just over 5 minutes.

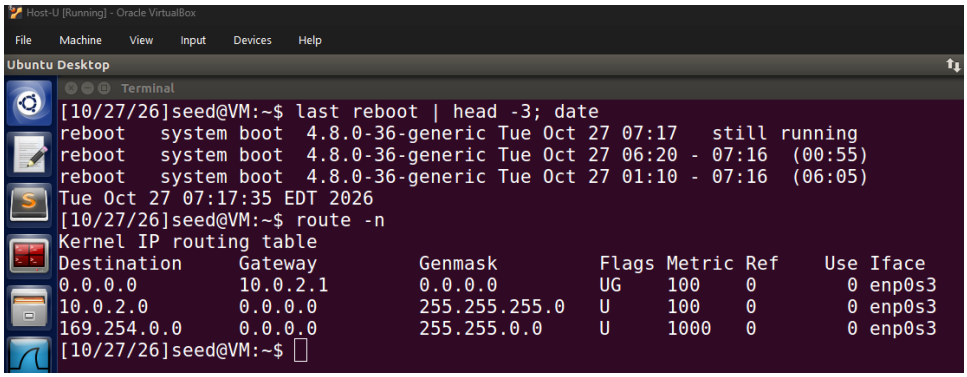


Figure 47: Host U: reboot at 07:17, current time 07:17:35.

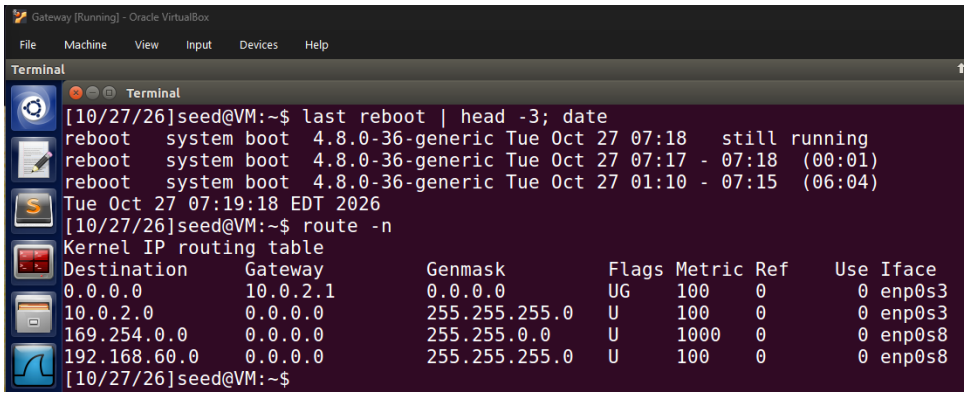


Figure 48: Gateway: reboot at 07:18, current time 07:19:18. Its routes are in place after the restart.

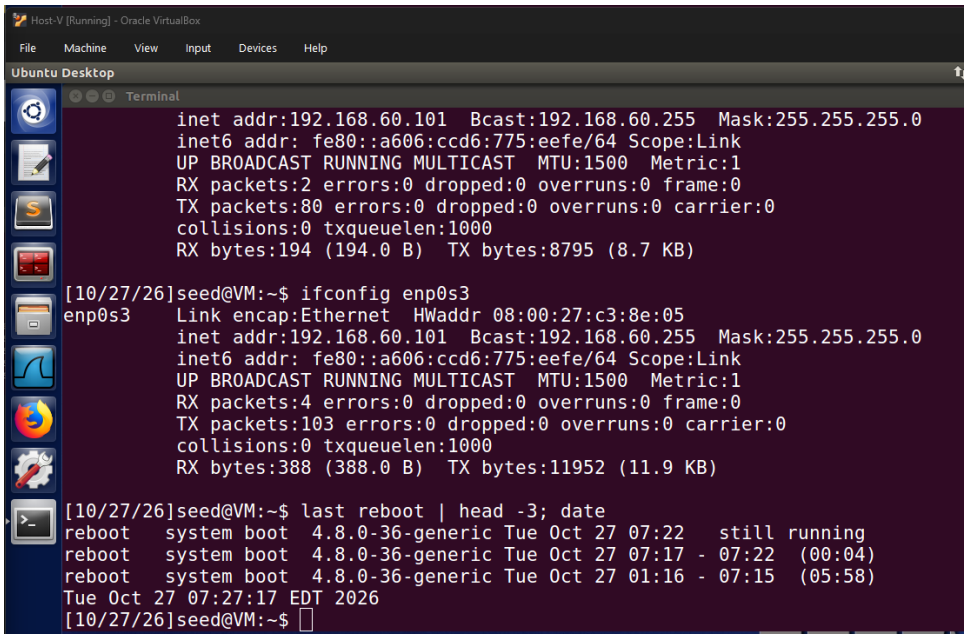


Figure 49: Host V: the address 192.168.60.101 is still there after the reboot.

Conclusion

In this lab, I built miniVPN, starting with a plain UDP tunnel and ending with a secure VPN. I used TLS to hide the traffic, and Wireshark confirmed that the pings and telnet inside the tunnel could no longer be read. The server proved who it was with a certificate signed by my own CA, and the client refused both the wrong name and a fake server, so no password was ever sent to an attacker. Only users with a valid account on the Gateway could open a tunnel. The server also handled two clients at once by giving each one its own tunnel address and its own child process. The tunnel-breaking and large-packet tests showed that the telnet session lives between the two computers rather than inside the tunnel, and that big packets are split up and put back together without loss. Overall, the lab showed that a VPN needs more than a tunnel: it also needs encryption, authentication of the server, and authentication of the user.